sPAR: (Somewhat) Practical Anonymous Router

Debajyoti Das[1] and Jeongeun Park[2]

[1] Lund University, Lund, Sweden
debajyoti.das@eit.lth.se
[2] Norwegian University of Science and Technology (NTNU), Trondheim, Norway
jeongeun.park@ntnu.no

Abstract. Anonymous communication is one of the fundamental tools to achieve privacy for communication over the internet. Almost all existing design strategies (e.g., onion routing/Tor, mixnets) for anonymous communication rely on the existence of some honest server/router in the network infrastructure to provide anonymity. A recent seminal work by Shi and Wu (Eurocrypt 2021) proposes the first cryptographic design for a *non-interactive anonymous router* (NIAR) that can use a single untrusted server or router to permute a set of messages without revealing the permutation to the untrusted router. This work is a really important step towards showing the possibility of designing such protocol from standard cryptographic assumptions. However, their construction is only of theoretical nature and still leaves many open questions towards realizing such systems in practice: (1) the cryptographic building blocks (multi-client functional encryption, correlated pseudorandom function) used in their design are really difficult to implement in practice. (2) Their setup phase takes the permutation as an input to generate the encryption/decryption keys; which means that the messages from the same sender in different *rounds* will be at the same position in the output vector, unless the setup phase is run before every round with a new permutation. (3) It is not known how to realize such a setup procedure, that initializes a random permutation obliviously, without any trusted entities in the system.
In this paper, we propose the first (somewhat) practical design, which we call *sPAR*, that solves the above problems using homomorphic encryption techniques. Our design also relies on a one-time setup phase, however the setup phase does not take any specific permutation as input. Instead, our design generates a fresh permutation for every round based on the random values locally generated by the clients. Already existing practical instantiations of fully homomorphic encryption (FHE) schemes make our design implementable and deployable in practice. Our design presents a new direction for designing anonymous communication systems. Unlike some existing systems like Tor, sPAR does not scale to millions of users, however, we demonstrate with a proof-of-concept implementation that sPAR could easily support around hundred users with a few seconds of latency for each message.

1 Introduction

Many anonymous communication systems [30,32,65,12,1] have been proposed in the last four decades with the goal to protect the privacy for users over the internet. Most design techniques rely on the existence of some honest parties in their infrastructure towards achieving their privacy goal. For instance, messages in mixnets [43,58,44,42,41] or Tor [30] need to go through at least one honest server on their paths to be 'mixed' with other messages. There exist designs based on multi-party-computation-based approaches (e.g., multi-server private information retrieval (PIR) write [22,32,31], multi-server oblivious message retrieval (OMR) [37,50].

A recent work by Shi and Wu [62] is the first work exploring the possibility of designing anonymous communication systems without requiring any honest component in the infrastructure. They show how to realize function-hiding Multi-Client Functional Encryption (MCFE) from a regular MCFE for *selection* function, and use that (in conjunction with correlated pseudorandom functions) to achieve *permutation* on an untrusted server or router, without the server learning the permutation. They call their design *Non-interactive Anonymous Router* or NIAR. In order to instantiate the function-hiding MCFE, the setup phase of their protocol takes the *permutation* as an input, and then many rounds of shuffles can be executed using the same setup. We discuss the difficulties to instantiate their solutions in Section 2.2 in detail.

Although their work takes the fundamental step towards showing the theoretical possibility of designing such systems, it still leaves many questions towards realizing them in practice unanswered:

1. How to generate the permutations, that the setup phase in NIAR takes as input, without any trusted component? And, how to realize the setup phase in practice so that the permutation is not leaked to anyone?
2. Is it possible to design an *oblivious permutation* algorithm with cryptographic building blocks that are easier to implement compared to function-hiding MCFE and correlated pseudorandom functions?
3. How practical can such designs be for real-world applications?

Overall, It still remains unknown if it is possible to provide a practical design for *anonymous routers*.

1.1 Our Construction

We take a positive step towards answering the above questions by designing an anonymous router based on homomorphic encryption techniques. We do not only show how to generate the permutation without any trusted setup, but also completely disassociate it from the setup phase. In our design each round of message-shuffle uses a fresh permutation based on the random values locally generated by the users. As a tradeoff, our protocol uses two rounds of client-server interactions for each message-shuffle as compared to one-round in NIAR. Additionally, we provide practical defense strategies against relevant active attacks for our protocol. Already existing practical instantiations of fully homomorphic encryption (FHE) schemes [17,18,33,11,10] allow us to implement our design with practical overheads for small number of users (a few seconds of latency for messages with a 128 users in the system). Since our design strategy is completely different from NIAR or any other existing anonymous communication systems, it presents a new avenue for designing such systems.

Challenges And Key Ideas At the core of our design is the use of multi-party FHE [3,52,55] which allows multiple parties to compute the same function over different messages without revealing each input to one another, by using a jointly generated public key, however the corresponding secret key is not known to anyone. We stress that employing FHE in the NIAR work has been avoided since using the same secret key for all clients would easily break anonymity (addressed in [62]), even though FHE could easily provide non-interactivity to the protocol by default. However, we could overcome the obstacle by using multi-party FHE[3] where each party has its own secret key to decrypt the message, however adding one round communication is necessary for decryption among multiple parties who possess distinct secret keys. Thanks to oblivious computation (homomorphic computation), the server executes permutation of messages without knowing the permutation encrypted under the joint key of all clients. Multi-party FHE comes with a one-time setup requirement, and the same setup can be used for many rounds of computations. However, we still need to generate a fresh permutation in every round.

Towards generating fresh permutations in every round, we use the following strategy: the clients generate random values locally, independent of all other clients and any past communications; the server can derive the permutation (obliviously) by using those (encrypted) random values received from the clients. More specifically, let us assume that the server holds an empty database with $\mathfrak{N}$ slots, where $\mathfrak{N} >> \eta$. A client with a message m can choose a random index $i \in \{1, \ldots, \mathfrak{N}\}$ and send encrypted m and i to the server. The server can place m at the position i without knowing either of the values with Homomorphic Placing [20]. After all the clients have placed their messages the same way, if i values are chosen uniformly at random by the clients, the ordering among the messages in the database (excluding the empty slots) would be a random permutation (assuming there is no collision). Then the server can launch the decryption phase to decrypt the messages.

However, unless $\mathfrak{N} \in \omega(\eta^2)$, there is a high probability of collision which results in a *failure* in the message delivery. Large $\mathfrak{N}$ would introduce a very high computation overhead for the server. To solve the

[3] We note that multi-key FHE [47] can also provide the same solution, however its ciphertext size grows linearly in the number of parties, hence we choose multi-party FHE for more practical purpose.

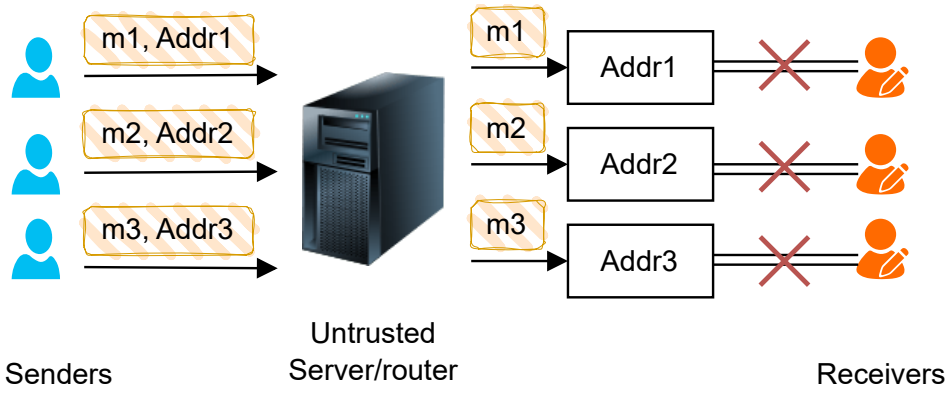

Fig. 1: System Model. We have only one untrusted server (untrusted for privacy, but trusted for availability). Each receiver is associated with an address, and the address should not leak the receiver's real identity. A sender wants to send a message m_i to the receiver associated with address Addr_i. The sender sends (m_i, Addr_i) in an encrypted format to the server; the server can deliver the message m_i to the receiver without knowing the association between the address and the physical identity of the receiver. The senders and the recipients do not trust each other, and all communications happen through the server/router.

collision problem without introducing large overhead, we use the trick of choice-of-three from balls-and-bin problem [51]: a client chooses three random indices, and the message is placed in the first available index/bin. We make the required modifications in the existing Homomorphic Demux [20] to enable choice-of-three. This choice-of-three brings down the failure probability to 10^{-11} with only $\mathfrak{N} = 3\eta$. However, this is an imperfect solution: this allows the adversary a small probability of finding the partial order between two messages under collision. To solve that problem, we sort each bin individually. Sorting each bin (with only three elements) does not add significant overhead.

These techniques allow us to provide a simple yet effective design for anonymous router that is practical for several small-scale applications. However, multi-party FHE comes with the requirement of a partial decryption step; and that adds one additional round of communication between the server and the clients. However, the clients still do not need to interact with each other.

Application Scenarios And Limitations. Our experimental evaluations up to 128 users and a latency of a few seconds demonstrate that sPAR can be used in practice for small-scale applications like mixing transactions in Cryptocurrencies (most existing solutions [60,35,9] in that space supports only up to a few hundred users), peer-to-peer messaging in a LAN network etc. We want to highlight that this work presents a first design of its kind, and leaves a lot of scope for future improvements in terms of performance as well as scalability. Additionally, our current work does not consider many possible issues related to real-world deployment scenarios: (i) clients could arbitrarily drop from the systems; (ii) different clients could send (or receive) messages at different rates in practice; and many more. We discuss about some of those possible avenues and future works in Section 7.

2 Technical Overview

2.1 Notations and System Model

We consider a system with a set of senders $\mathcal{S}$ that want to send messages to a set of recipients $\mathcal{R}$ via an intermediate router P. In real life, the same user can act as sender as well as recipient, however, we consider

the sender role and recipient role as two separate logical entities. Each sender is denoted by u_i where $i \in \{1, \ldots, \eta\}$ and $|\mathcal{S}| = \eta$. Similarly, each recipient is denoted by r_i where $i \in \{1, \ldots, \eta'\}$ and $|\mathcal{R}| = \eta'$. We summarize all the system parameters in Figure 2.

We consider that our protocol works in *rounds*: in every round, the server/router collects all the incoming messages, runs the required computations as well as decryption mechanism, and then forwards the messages to the intended recipients. For simplicity we assume that $\eta = \eta'$, and each sender sends exactly one message in every round. This setting is also similar to that of NIAR [62] and many other designs for anonymous communication [26,8,34,23,67,43]. We discuss the possible relaxations in Section 7.

Similar to NIAR, we consider the setting where the clients do not communicate with each other directly: all the communications go through the router/server. We want to provide anonymity for both senders and recipients. First we present a version that achieves *sender anonymity*, i.e., the server can retrieve all the (shuffled) messages in plaintext and then deliver to the intended recipients. Then we provide further addendum to the protcol towards achieving *recipient anonymity* even from a compromised sender colluding with one of the servers.

η	Total number of senders
η'	Total number of recipients
$\mathcal{S}$	Set of all senders $u_i \forall i \in \{1, \ldots, \eta\}$
$\mathcal{R}$	Set of all recipients $r_i \forall i \in \{1, \ldots, \eta\}$
ℓ	Maximum latency allowed for a message
P	untrusted router
λ	The security parameter
δ	the adversarial advantage
$\xleftarrow{\$} [b, c]$	Draw uniformly at random from $[b, c]$

Fig. 2: Protocol and system parameters for sPAR

2.2 Existing Works And Their Limitations

Most design strategies for anonymous communication systems (e.g., Tor [30], mixnets [29,43,42], MPC-based designs [31,32,37]) explicitly require at least one honest party in their infrastructure. We skip a detailed discussion about them and refer to some excellent existing works [61,25,63,29] that provide surveys on such systems.

NIAR We mainly compare our design with the work of NIAR [62] by Shi and Wu, since this is the only work that achieve anonymity without any trusted component in the infrastructure. This was a breakthrough work to show the theoretical possibility of designing *anonymous routers*. They also show the possibility of achieving non-interactive designs. This work provides a very crucial insight: such protocols need to realize *oblivious permutation* function on the untrusted server.

However, there seems to be many obstacles towards making their design suitable for practical use: (1) The instantiations of the cryptographic building blocks (function-hiding MCFE and correlated pseudorandom function) are extremely expensive, if implementable. (2) The version of their protocol that additionally protects the anonymity for recipients (they call this property *full-insider protection*) requires sub-exponentially secure indistinguishability obfuscation — the practicality of such assumptions is unclear. (3) Additionally, the setup phase of their protocol takes the *permutation* as an input, and then many rounds of shuffles can be executed using the same setup. All the messages from the same sender in between setup-phases are linkable to each other even though the server does not know the permutation, unless the setup phase is run before every round with a new permutation. This creates a tradeoff between the *non-interactive* property and strong anonymity guarantee.

Our design sPAR relaxes the requirement of being *non-interactive*: it requires only two rounds of client-server communication, and no other communication among the clients. sPAR relies on RLWE assumption and cryptographic security of hash functions, easily implementable, and can be suitable for applications with a few hundred users (e.g., mixing transactions on blockchain, private messaging in a LAN network).

DC-nets. Dining cryptographers networks or DC-nets are very closely related to our design in terms of scalability and application scenarios. Some DC-net designs can scale for a few hundred users, and used in the cryptocurrency or LAN scenario. Peer-to-peer DC-nets [34,60,23] even achieve similar anonymity guarantees without requiring any external trusted parties. However, their communication model is very different from our scenario: they require active interactions between each pair of clients to execute the protocol. Other designs require a separate set of MPC servers helping the protocol [67,22,32,1,48,24,8,26] to avoid the interactions among the clients.

Designs Based on Trusted Execution Environments (TEEs). There are some recent designs that utilize Trusted hardware (e.g., Intex SGX) to design anonymous communication systems [50]. An untrusted server/router use the trusted hardware component to shuffle the messages without being able to read the messages to decifer the permutation. However, this requires trust assumption on the hardware component which is known to fail in many cases [39,46].

2.3 Security Requirements And Definitions

Our security goals are similar to that of NIAR [62]. However, instead of using their definitions, we use indistinguishability-based definitions of *sender anonymity* and *recipient anonymity* that are close to and borrowed from a long line of literature on anonymous communication systems [38,43,29,26,13,42,41] and their formalization [28,27,40,7]. This allows us to be comparable with other design paradigms of anonymous communication in terms of anonymity guarantees.

The choice is also because of a weakness in the definitions of NIAR [62]: if a mixing/anonymity protocol uses the same permutation for multiple rounds, a compromised recipient will know that she is receiving all the messages from the same sender, even though she does not know who that sender is. And that leakage might help the adversary build a strong profile about the sender. The definition considered by Shi and Wu does not consider that leakage, and only considers the leakage about the permutation used in the protocol.

Our protocol achieves sender anonymity: which is analogous to the *receiver-insider protection* defined in the NIAR paper. Additionally, we extend our protocol to achieve recipient anonymity (even from corrupted senders colluding with the server) — this is analogous to their *full-insider protection* notion. Moreover, the definitions we use account for the leakage mentioned above. We present the definitions below.

Sender Anonymity Game The sender anonymity game, denoted by $\mathcal{G}_{\mathsf{SA}}^{\Pi,\mathcal{A},\eta}(1^\lambda)$, between a challenger $\mathtt{Ch}$ and an adversary $\mathcal{A}$ for protocol Π can be described as follows:

- The challenger $\mathtt{Ch}$ provides the adversary $\mathcal{A}$ with the description of Π.
- $\mathcal{A}$ statically corrupts all senders in $\mathcal{S}$ except from a pair u_0, u_1, all recipients, and the server P .
- $\mathtt{Ch}$ and $\mathcal{A}$ engage in an execution of Π where $\mathtt{Ch}$ acts on behalf of u_0 and u_1, while $\mathcal{A}$ controls the corrupted parties and monitors the network traffic as a global passive adversary.
- Before any round of choice the $\mathcal{A}$ can send a pair of challenge messages (u_0, r^*, m_0) and (u_1, r^*, m_1) to $\mathtt{Ch}$. Here r^* is a corrupted recipient. where u is the sender of the message and m the content.
- In turn, $\mathtt{Ch}$ chooses a random bit $b \in \{0, 1\}$ and initiates the challenge transmissions in that round according to the following cases:
  - If $b = 0$, Π transmits the messages (u_0, r^*, m_0) and (u_1, r^*, m_1).
  - If $b = 1$, Π transmits the messages (u_1, r^*, m_0) and (u_0, r^*, m_1).
- $\mathcal{A}$ can terminate the game any time by outputting a bit b^*, as a guess for the challenge bit b.
- The game returns 1 if and only if $b^* = b$ (i.e., $\mathcal{A}$ guesses correctly), otherwise the game returns 0.

Definition 1 (Sender Anonymity). *A protocol Π provides* sender anonymity *for messages up to probability δ for $0 \leq \delta < 1$ if, for all probabilistic polynomial time (PPT) adversaries $\mathcal{A}$, and the security parameter λ the following holds:*

$$\Pr\left[\mathcal{G}_{\mathsf{SA}}^{\Pi,\mathcal{A},\eta}(1^\lambda) = 1\right] - \Pr\left[\mathcal{G}_{\mathsf{SA}}^{\Pi,\mathcal{A},\eta}(1^\lambda) = 0\right] \leq \delta(\lambda).$$

For our purposes, we want $\delta()$ to be negligible in λ.

Recipient Anonymity Game. The recipient anonymity game $\mathcal{G}_{\mathsf{RA}}^{\Pi,\mathcal{A},\eta'}(1^\lambda)$ between a challenger $\mathtt{Ch}$ and an adversary $\mathcal{A}$ for protocol Π can be described as follows:

- The challenger $\mathtt{Ch}$ provides the adversary $\mathcal{A}$ with the description of Π.
- $\mathcal{A}$ statically corrupts all recipients in $\mathcal{R}$ except from a pair r_0, r_1, all senders, and the server P .
- $\mathtt{Ch}$ and $\mathcal{A}$ engage in an execution of Π where $\mathtt{Ch}$ acts on behalf of r_0 and r_1, while $\mathcal{A}$ controls the corrupted parties and monitors the network traffic as a global passive adversary.
- Before any round of choice the $\mathcal{A}$ can send a pair of challenge messages (u^*, r_0, m_0) and (u^*, r_1, m_1) to $\mathtt{Ch}$ where r is the intended recipient of the message and m the content.
- In turn, $\mathtt{Ch}$ chooses a random bit $b \in \{0, 1\}$ and initiates the challenge transmissions in that round according to the following cases:
  - If $b = 0$, Π transmits the messages (u^*, r_0, m_0) and (u^*, r_1, m_1).
  - If $b = 1$, Π transmits the messages (u^*, r_1, m_0) and (u^*, r_0, m_1).
- $\mathcal{A}$ can terminate the game any time by outputting a bit b^*, as a guess for the challenge bit b.
- The game returns 1 if and only if $b^* = b$ (i.e., $\mathcal{A}$ guesses correctly), otherwise the game returns 0.

Definition 2 (Recipient Anonymity). *A protocol Π provides* recipient anonymity *for messages up to probability δ for $0 \leq \delta < 1$ if, for all probabilistic polynomial time (PPT) adversaries $\mathcal{A}$, and the security parameter λ the following holds:*

$$\Pr\left[\mathcal{G}_{\mathsf{RA}}^{\Pi,\mathcal{A},\eta}(1^\lambda) = 1\right] - \Pr\left[\mathcal{G}_{\mathsf{RA}}^{\Pi,\mathcal{A},\eta}(1^\lambda) = 0\right] \leq \delta(\lambda).$$

2.4 Protocol Overview

Our protocol has one setup phase, and then the server can run polynomially many number of *rounds* using the same setup.

Setup Phase. As part of the setup phase, each client produces their individual secret key s_i, and a collective encryption key (the common public key) $\hat{\mathsf{pk}}$. Any message m encrypted with the common public key to generate $c = \mathsf{Enc}_{\hat{\mathsf{pk}}}(m)$ can only be decrypted if all the clients provide a *partial decryption* using their corresponding secret keys. However, the master secret key corresponding to $\hat{\mathsf{pk}}$ is not known to anyone.

Online Phase: Protocol Round Similar to the setting of NIAR [62], we consider batch-based message processing: each client sends exactly one message during a protocol *round*. After collecting messages from all the clients, the server starts the decryption procedure for that round. In case the client does not have a message to send, the client can send a random message.

At the beginning of a round, the server maintains η buckets, each bucket containing three slots (so a total of 3η slots). The client u_i with a message m_i picks three indices a_1, a_2, a_3, each from $\{1, \ldots, \eta\}$ uniformly at random and independent of each other. Then the client sends encrypted m_i, a_1, a_2, a_3 to the server. Then the server homomorphically places the message in the least filled bucket among the buckets corresponding to a_1, a_2, a_3 with a technique similar to Homomorphic Demux [20]; and the server does not learn anything about the message m_i or the values of a_1, a_2, a_3. We call this step $\mathsf{HomPlacing}$.

After the server has received and placed messages from all the clients, the server sorts each bucket individually. Then it starts the decryption step: the server sends the encrypted message vector to the clients for partial decryption. After receiving the partial decryptions from the clients, the server can decrypt the messages by aggregating them. The partial decryption per each client is a vector of $O(\eta)$ RLWE ciphertext.

Note that, without the sorting step, there is a small leakage to the adversary. Whenever the messages placed in the same bucket maintain their partial order without the sorting step. That means the server can distinguish the senders of a pair of messages placed in the same bucket with non-negligible probability. Since, each bucket is of size three, the sorting does not add significant overhead; and this step is highly parallelizable. Assuming enough randomness in message values, we show that this version of protocol provides sender anonymity with negligible adversarial advantage.

Why Not Sorting the Whole Set Instead of HomPlacing With the assumption that messages come from a uniform distribution, sorting can be used for shuffling a set of messages. However, the concrete computation time for sorting encrypted data scales really poorly with the number of elements. The sorting algorithm [21] we use for sorting the buckets would take 8913 seconds [6] to sort a list of 128 elements. One round of our protocol requires less than a few seconds to complete (c.f. Section 5.2).

Packing Optimization In order to reduce the communication cost, we make use of a well-known packing method of RLWE ciphertext [15] converting η RLWE ciphertexts into an RLWE ciphertext encrypting a polynomial of η messages embedded into its coefficients, therefore we can reduce the size of partial decryption down to only one ciphertext when the polynomial degree $N > \eta$.

Recipient Anonymity We also extend our protocol to provide anonymity for the recipients. The recipient anonymity property we consider (c.f. Section 4.4 and Definition 2) is similar to the flavor of onion services (formerly 'hidden services') in Tor [36]. The sender might know a *public-identity* (e.g., onion address in the context of Tor, or a public-key) for the intended recipient, however, does not know the actual/physical identity r_i of the recipient hidden behind that public identity. Recipient anonymity in conjunction with sender anonymity is analogus to *full-insider protection* notion used in NIAR.

To that end, we consider that each recipient $r_i \in \mathcal{R}$ has a public-key $\mathsf{pk}_i^{\mathsf{classical}}$ associated with them, and only r_i knows the corresponding secret key $\mathsf{sk}_i^{\mathsf{classical}}$. We employ a trivial version of oblivious message retrieval (OMR) to achieve anonymity for recipients. The sender of a message uses two layers of encryption for a message m: (1) first the m is encrypted with $\mathsf{pk}_i^{\mathsf{classical}}$ to generate m', and then (2) $\mathsf{Enc}_{\hat{\mathsf{pk}}}(m')$ is sent to the server. After the decryption phase, the server broadcasts the output of the decryption phase to all the recipients. Only the intended recipient can decrypt m' to retrieve the message m.

2.5 Active Attacks

We do not consider active attacks from the server solely for the purpose of denial-of-service (DoS). However, we consider active attacks with the purpose of breaking anonymity, which could potentially go undetected by honest clients. We want to highlight that all corrupted parties can collude with each other in our threat model.

The relevant active attacks we consider in our system are following: (i) equivocation and (ii) targeted message drops by the server, as well as (iii) targeted disruptions from malicious clients by sending malformed ciphertexts.

Considering that our design relies on FHE, there are well-known techniques to achieve verifiable FHE [5,66] that would allow the server to prove correctness of the computation to the honest clients. However, we aim to achieve more efficient techniques towards protecting against active attacks. We defend against equivocation attack by incorporating the hash of the history of communication in the public-key used in the multi-party FHE scheme, and we do so without using any additional round of communication.

Then we provide reactive defense mechanism in the presence of targeted active attacks: the honest clients can detect such attacks with high probability without adding any significant overhead to the performance. When such attacks are detected, we provide a *verification* mechanism to detect the disruptive party. We refer to Section 6 for the detailed description.

Table 1: Parameters related to the FHE scheme

$\mathbf{x}_i, \mathbf{x}[i]$	The i-th component of vector $\mathbf{x}$
$\log(\cdot)$	Logarithm function with base 2
N	The maximum degree of a polynomial
t	Plaintext modulus
q	Ciphertext modulus
ℓ	Decomposition parameter
R	$\mathbb{Z}[X]/(X^N+1), N \in \mathbb{N}$
R_q	$\mathbb{Z}[X]/(X^N+1) \mod q, q, N \in \mathbb{N}$
λ	Security parameter

3 Building Blocks

3.1 Preliminaries of Underlying FHE Scheme

Our protocol, introduced in the next section, can be instantiated with any IND-CPA secure FHE scheme that supports the operations presented in this section. However, for concrete instantiation, we chose a scheme which is secure based on the (R)LWE problem [59,49], and supports the efficient homomorphic operations that we define below.

Notations We denote λ as the security parameter of the FHE scheme. We define vectors and matrices in lowercase bold and uppercase bold, respectively. For a vector $\mathbf{x}$, both $\mathbf{x}_i$ and $\mathbf{x}[i]$ denote either the i-th component scalar or the i-th element of an ordered finite set. $\log(\cdot)$ is the logarithm function with base 2. Let R and R_q denote $\mathbb{Z}[X]/(X^N+1)$ and $(\mathbb{Z}/q\mathbb{Z})[X]/(X^N+1)$, respectively, for positive integers q and N. We summarize our notations in Table 1.

We introduce ring learning with error (RLWE) problem which we are relying on for the security of an FHE scheme that we need to instantiate our protocol.

Definition 3. *For security parameter λ, let $\Phi_m(X)$ be a cyclotomic polynomial with degree $\phi(m)$ and set $\mathcal{R} = \mathbb{Z}[X]/(\Phi_m(X))$. Let $q = q(\lambda) \geq 2$ be an integer. For a fixed secret random element $s \in \mathcal{R}_q$ and a distribution $\chi = \chi(\lambda)$ over $\mathcal{R}$, the decisional RLWE problem says the distribution outputting $(a, a \cdot s + e)$, where a and e is chosen uniformly at random from $\mathcal{R}_q$ and χ, respectively, and the distribution outputting (a, u) chosen uniformly at random from $\mathcal{R}_q^2$ are computationally indistinguishable.*

The sample $(a, a \cdot s + e) \in \mathcal{R}_q^2$ is called RLWE sample. For a simple and convenient use like many schemes, we set $\Phi(X) = X^N + 1$, where N is a power of 2.

Ciphertexts We define the ciphertext modulus as q and the plaintext modulus as t, where $t \ll q$, and denote $\Delta = \lfloor q/t \rceil$.

- An RLWE ciphertext is defined as $c := (a, b) \in R_q^2$, where $b = a \cdot s + \Delta \cdot m + e$ for a message polynomial $m \in R_t$ and a secret key $s \in \mathcal{R}$. c is denoted by $\mathsf{RLWE}_s(m)$.
- Given a base g and $\ell = O(\log q)$, we define a gadget vector $\mathbf{g} = (1, g, \ldots, g^{\ell-1})^t$. An RGSW ciphertext is a form of $\mathbf{C} := (\mathbf{a}, \mathbf{b}) \in R_q^{2\ell \times 2}$, where $\mathbf{b} = \mathbf{H} + m \cdot \mathbf{G}$, where each row of $\mathbf{H}$ is an $\mathsf{RLWE}_s(0)$ and $\mathbf{G}$ is a gadget matrix which is defined by $\mathbf{G} = \mathbf{I}_2 \otimes \mathbf{g}$. The ciphertext is denoted as $\mathsf{RGSW}_s(m)$. s can be omitted if it is clear from the context.

Below is the definition of homomorphic operations that we need for our protocol.

Homomorphic Operations and Basic Algorithms We introduce here the homomorphic operations and basic algorithms used in this paper. For now, we define operations in the context of single key case, however they can be extended to multi-party setting very easily by viewing the single key as the sum of all secret keys of all users.

Homomorphic Addition Let $c_1 := (a_1, b_1)$ and $c_2 := (a_2, b_2)$ be two RLWE ciphertexts. The addition between two ciphertexts is defined as $c_+ := c_1 + c_2 = (a_1 + a_2, b_1 + b_2)$.

External Product We denote a homomorphic multiplication between an RLWE ciphertext and an RGSW ciphertext as $\boxdot$, which is called external product in [18], and define it as follows: $a \boxdot \mathbf{B} = \mathbf{G}^{-1}(a) \cdot \mathbf{B}$, where $\mathbf{B}$ is an $\mathsf{RGSW}(m_B)$ where $B \in \{0, 1\}$ and a is an $\mathsf{RLWE}(m_a)$ where $m_a \in R_t$ and $\mathbf{G}^{-1}(\cdot)$ is the gadget decomposition function which satisfies $\mathbf{G}^{-1}(a) \cdot \mathbf{G} = a + e \mod q$ for $a \in R_q^2$, where e is some error introduced by the decomposition.

$\mathsf{HomExpand}$*:* This algorithm takes an RLWE ciphertext encrypting a message, and outputs an RGSW ciphertext encrypting the same message. We use this algorithm to generate an RGSW ciphertext from an RLWE ciphertext which is given by a client. Since RGSW ciphertext is bigger than RLWE ciphertext, clients are recommended to send an RLWE ciphertext packing all necessary messages and let server unpack and turn them into RGSW ciphertexts to reduce the bandwidth of the protocol. We refer to [14,20] for readers to see how it works in detail.

3.2 Multi-Party Homomorphic Encryption

Multi-Party homomorphic encryption (a.k.a Threshold multikey homomorphic encryption) [4,52] requires a setup process before the computation among a fixed number of parties; generating a common public key requires only one round [55]. That is, there should be a fixed number of participants before a computation starts, then all of them need to generate a common public key to encrypt their inputs, hence the ciphertext size is the same as single key homomorphic encryption. The common key generation is done via only one round of communication of all users, and the key does not reveal any information of each user's secret key.

Once the common key is generated among all users, the rest except decryption is the same as the single key FHE, for instance, encryption, homomorphic operations. The decryption becomes a protocol among all parties taking one round to compute the same computed value, which is discussed below. We consider the following functions related to a Multi-Party Homomorphic Encryption scheme:

- $\mathsf{MPSetup}$: This step is run collectively by all p parties to generate a common public key. At the end of the step, all participants obtain the same public key denoted by $\hat{\mathsf{pk}}$, without knowing the underlying secret key denoted by $\hat{\mathsf{sk}} := \sum_{i=1}^{p} \mathsf{sk}_i$, where sk_i is the i -th client secret key.
- $\mathsf{Local.Enc}_{s_i}(m)$: Encryption of m under the key s_i of the i-th party, which produces a ciphertext c under its own key s_i.
- $\mathsf{Global.Enc}_{\hat{\mathsf{pk}}}(m)$: Encryption of m under the common key $\hat{\mathsf{pk}}$.
- $\mathsf{Eval}(F, c_1 = \mathsf{Enc}_{\hat{\mathsf{pk}}}(m_1), c_2 = \mathsf{Enc}_{\hat{\mathsf{pk}}}(m_2), \dots)$: Evaluation of the function $F(m_1, m_2, \dots)$ by operating on the ciphertexts $c_1, c_2, \dots$, it produces a ciphertext c that encrypts the output of $F(m_1, m_2, \dots)$ under the common public key $\hat{\mathsf{pk}}$.
- $\mathsf{Dec}(\mathsf{sk}_1, \dots, \mathsf{sk}_p, F, c)$: Decryption of a ciphertext c that is the result of $\mathsf{Eval}(\mathsf{F}, \dots)$ operation. This step requires all the keys $\mathsf{sk}_1, \mathsf{sk}_2, \dots, \mathsf{sk}_p$, (in fact, the secret key holders should join the decryption.) for the ciphertexts that have been used to generate the common public key $\hat{\mathsf{pk}}$. This step is divided into two sub-phases:
  - $\mathsf{Part.Dec}(\mathsf{sk}_i, c)$: Partial decryption of a ciphertext c by a client using their own secret key sk_i. And distribute the result to all the parties.
  - $\mathsf{Fin.Dec}(c)$: Final decryption of a ciphertext c by a client to recover the underlying message $F(m_1, m_2, \dots)$ by either aggregating all the given p partial decryptions or decrypting them by its own secret key depending on the target security model.

Multi-party FHE based on RLWE Now we introduce an instantiation of a multi-party FHE scheme based on RLWE, adapted from [55] below. We use the same parameters and ciphertexts defined Section 3.1.

– $\mathsf{MPSetup}(1^\lambda) \to (\mathsf{params}, \mathsf{sk})$: It takes security parameter λ and outputs public parameter $\mathsf{params} = \{N, \chi, \chi', q, \Delta, a\}$, where N is a polynomial degree, χ, χ' are secret key/error distributions, and $a \in \mathcal{R}_q$ is a common random parameter (called a CRS) which all users are given. params is implicitly included in every algorithm below as an input. We note that p is the number of parties involved in the computation. Each party runs this setup and generates its own secret key $\mathsf{sk} = s_i$.
– $\mathsf{Local.Enc}_{s_i}(0) \to \mathsf{pk}_i$: Given a secret key s_i, and a message 0 to generate its public key $\mathsf{pk}_i := (a, b_i = as_i + e_i)$, by using the CRS a and $e_i \leftarrow \chi$.

After the key generation of each user locally, all users share their public keys through a public channel. Then users generate a common public key by adding the second component of all given public keys. We denote the common public key by $\hat{\mathsf{pk}} = (a, \sum_{i=1}^{p}(b_i))$ such that $a(\sum_{i=1}^{p} s_i) + b \approx 0 \mod q$. Therefore, the corresponding secret key to $\hat{\mathsf{pk}}$ is the sum of all users' secret key $\sum_{i=1}^{p} s_i$, which we call the master secret key, and it is not known to anyone. In fact, no one can recover the master secret key from $\hat{\mathsf{pk}}$.

– $\mathsf{Global.Enc}_{\hat{\mathsf{pk}}}(m_i) \to \mathsf{ct}_i$: The i-th party encrypts its message m under the common public key $\hat{\mathsf{pk}}$ and compute the output ct_i like the following:
  • parse $\hat{\mathsf{pk}} := (a', b')$ such that $a' \cdot (\sum_{i=1}^{p} s_i) + b \approx 0 \mod q$.
  • samples $u \leftarrow \chi, e_1, e_2 \leftarrow \chi$
  • computes a fresh ciphertext $\mathsf{ct}_i = (ua' + e_1, ub' + e_2 + \Delta m) \in \mathcal{R}_q^2$.
  • Outputs ct_i.
– $\mathsf{Dec}(\mathsf{sk}_1, \ldots, \mathsf{sk}_p, \mathsf{ct}) \to m$: On input all users' secret keys $\{\mathsf{sk}_i = s_i\}_{i\in[p]}$ and a ciphertext ct. The partial decryption protocol is defined like the following:
  • $\mathsf{Part.Dec}(\hat{\mathsf{ct}}, \mathsf{sk}_i) \to d_i$: It takes a common evaluated ciphertext $\hat{\mathsf{ct}} : (\hat{a}, \hat{b})$ where $\hat{b} = \hat{a}(s_1 + s_2 + \cdots + s_p) + \Delta m + e$ and i-th user's secret key sk_i on input. It returns a partial decryption $d_i := \hat{a}s_i + e_i$ where $e_i \leftarrow \chi'$.
  • $\mathsf{Fin.Dec}(\hat{\mathsf{ct}}, \{d_i\}_{i\in[p]})$: It takes all the partial decrypted messages $\{d_i\}_{i\in[p]}$ that are given by all parties. It outputs the correct evaluated message by computing $m := \frac{1}{\Delta}(\hat{\mathsf{ct}}[2] - \sum_{i=1}^{p} d_i)$.

Now with the definition above, we can allow homomorphic operations that we need for our protocol, defined in Section 3.1, by viewing ct_i as an RLWE ciphertext under $\hat{\mathsf{pk}}$, and forming an RGSW ciphertext as defined in Section 3.1 consisting of RLWE samples encrypting 0 under $\hat{\mathsf{pk}}$.

Security of multi-party FHE We refer to [55] for IND-CPA security of our instantiation. The main goal for each party is to protect its own message encrypted under the common key $\hat{\mathsf{pk}}$. IND-CPA security of multi-party FHE guarantees that the common public key does not reveal any information of each party's secret key and a ciphertext encrypted under the common public key does not reveal any information about the underlying message.

3.3 Homomorphic Placing

In order to place each message to the right position (corresponding to the target recipient), we use homomorphic traversal algorithm introduced in [20], to place an item to the designated component of the output vector homomorphically, while the rest components are encryptions of 0. We introduce the algorithm adapted to our protocol (See Algorithm 1).

The algorithm starts with the root of an empty but complete tree of depth $L := \log n$ with n leaves. It takes an encrypted value V_r and $\log n$ encrypted bits $\{c_0, \ldots, c_{s-1}\}$. Then V_r is put to the root of the tree, while other nodes are filled with 0 (remaining empty).

The encrypted $\log n$ bits indicates the address of the position where the other input value (denoted by V_r) will be travelled to. For example, if the input bits are a bit representation of 3, then the input value V_r will be copied to the third leaf from the left (we count from 0). To do this, each node should evaluate an operation to keep all the values oblivious to the computing party, therefore the computation complexity is $O(n)$. Each computation of each left/right child node is performed via homomorphic operation, therefore the $\cdot$ denotes

Algorithm 1 Homomorphic placing HomPlacing for one slot

Input: An encrypted value V_r, and $\log n$ encrypted bits $\{c_0, \ldots, c_{L-1}\}$, where $L = \log n$.
Output: a vector $\mathbf{l}$ consisting of n values of leaves: $z_0, \ldots, z_{n-1}$, where $\mathbf{l}[i] = z_i$ for $i \in \{0, ...n-1\}$
$\mathbf{l} := \mathbf{0}$
Initialize root: $b_0 = V_r$
Initialize the node values: $b_i := 0$ for $i \in \{1, \ldots, 2n-2\}$
for $i \leftarrow 0, \ldots, L-1$ **do**
    for $j \leftarrow 0, \ldots, 2^i - 1$ **do**
        $b_{2^{i+1}+2j-1} := b_{2^i-1+j} - b_{2^i-1+j} \cdot c_i$
        $b_{2^{i+1}+2j} := b_{2^i-1+j} \cdot c_i$
    end for
end for
for $i \leftarrow 0, \ldots, n-1$ **do**
    $z_i := b_{2^{L+1}-1+i}$
    $\mathbf{l}[i] = z_i$
end for
return $\mathbf{l}$

homomorphic multiplication. In our instantiation, we use external product to keep practical computation cost. We will use Algorithm 1 as a part of our anonymous router construction in the next section. When it is applied to our construction, we let clients choose multiple slots (indices) for their message to be placed to avoid collision. If the number of slots is the same as the number of clients, there is a high change for at least two clients to choose the same slot since they do not communicate to choose the slots. Therefore, we increase the size of slots by introducing additional parameter, so that we can guarantee the possibility of having collision between any two parties is negligible.

As a result, in our adapted algorithm, clients prepare multiple indices for such slots, there are additional loops for placing the value to the only one slot (see Algorithm 2).

4 Anonymous Router Construction

With this version of protocol, the server maintains η bins/buckets. Each user for each message picks three random indices, and the message is written to the bucket corresponding to one of the indices. The server obliviously chooses the least full bucket among the three to write the message, by operating on the homomorphic ciphertexts.

4.1 Setup

Similar to the previous section, we consider a round based communication model. In a round-based protocol, the clients send messages in a batch following rounds, and once that batch is processed completely, another new rounds starts and the clients can send messages again. In our design we have one setup phase followed by many communication rounds (polynomially bounded).

Setting up Multi-party FHE As part of setup each client u_i participates in a multi-party setup $\mathsf{MPSetup}$ to produce their individual secret key s_i, and a collective encryption key (the common public key) $\hat{\mathsf{pk}}$. Additionally, they all have the public key $\hat{\mathsf{pk}}$ that will allow them to generate $\mathsf{Enc}_{\hat{\mathsf{pk}}}(m)$ for a message m. We again stress that the corresponding master secret key to $\hat{\mathsf{pk}}$ is not known to anyone.

4.2 Online Phase

We consider that each client sends exactly one message in every round. The online phase occurs in the following four steps.

Server: Setting up the buckets If the are η senders, the server creates a matrix L of size $\eta \times 3$, each element in the matrix contains an encryption of 0. The server also creates an auxiliary list I of size $\eta \times 3$, each element is an encryption of 1. The value 1 for an element in I denotes that the corresponding position in L is available to write.

Client: Encryption Suppose each client u_i has the message m_i. The client executes the following steps:

- u_i encrypts m_i with the following: $c_i = \mathsf{Global.Enc}_{\hat{\mathsf{pk}}}(m_i)$.
- u_i picks three random indices a_1, a_2, a_3 such that $a_w \in [0, \eta-1] \forall w \in \{1,2,3\}$. u_i binarizes each a_w and encrypts their binary representations, denoted by $[a_w]_2$ with the following: $A_{i,w,j} = \mathsf{Global.Enc}_{\hat{\mathsf{pk}}}([a_w]_2[j])$,where $[a_w]_2[j]$ is the j-th bit of $[a_w]_2$ for $j \in \{0, 1, \ldots, \log \eta\}$.
- then u_i will send $c_i, \{\{A_{i,w,j}\}_{j\in\{0,1,\ldots,\log\eta\}}\}_{w\in\{1,2,3\}}$ (for convenience we use $\{\{A_i\}_j\}_w$ from now on, and $A_{i,w,j}$ for each individual encrypted bit) to the server using a public channel.

At the end of this phase, the server/router P receives η tuples $(c_1, \{A_{1,1}\}_j, \{A_{1,2}\}_j, \{A_{1,3}\}_j), \ldots, (c_\eta, \{A_{\eta,1}\}_j, \{A_{\eta,2}\}_j, \{A_{\eta,3}\}_j)$, where each tuple $(c_i, \{A_{i,1}\}_j, \{A_{i,2}\}_j, \{A_{i,3}\}_j)$ is received from the client u_i.

Server: Write The server holds the ciphertexts of the following form: $c_i = \mathsf{Global.Enc}_{\hat{\mathsf{pk}}}(m_i)$, $\{\{A_{i,w}\}_j\}_w$ for $i \in \{1, \ldots, \eta\}$. The server runs the following steps for each tuple $(c_i, \{\{A_{i,w}\}_j\}_w)$:

- $A'_{i,w,j} = \mathsf{HomExpand}(A_{i,w,j})$, to expand them to an RGSW ciphertext for each j-th bit of $[a_w]_2$, for $j \in \{0, ..., \log \eta\}$ and $w \in \{1, 2, 3\}$.
- runs $\mathsf{HomPlacing}(c_i, \{\{A'_i\}_j\}_w)$ for each $i \in [\eta]$. Then the list stored by the server is updated with the given message.

A pseudocode for the server's write process for each client-input is illustrated in Algorithm 2. After executing the above steps for each i, to hide the partial order of messages in the same bucket, the server obliviously *sorts* [21] each bucket individually after all the messages are placed. Then it broadcasts all the elements of L to all the clients. the server broadcasts all the elements of the list to all the clients.

We note that the protocol has no overflow with high probability due to the analysis discussed in Section 4.3. Additionally, if there is an error in message delivery because of an overflow, the client can simply retransmit the message without deteriorating anonymity.

Client: Partial Decryption Each client partially decrypts $\mathbf{u}$ using their own secret key s_i, by computing $d_i = \mathsf{Partial.Dec}(s_i, \mathbf{u})$. The client sends the partial decryption d_i to the server. We note that d_i is not one ciphertext but a vector of dimension $3 \times \eta$, where each component is an RLWE ciphertext.

Server: Combination and Final Decryption After the server receives the values $d_1, \ldots, d_\eta$, the server combines them by running $\mathsf{Fin.Dec}(d_1, \ldots, d_\eta)$ to obtain the shuffled $m_1, \ldots, m_\eta$ (the server discards the 0 values). The server then broadcasts all the messages.

Optimizing bandwidth We can optimize the communication complexity and the concrete communication cost by using a packing method. We first recall that the complexity of distributed messages over a public channel after partial decryption is $O(\eta^2)$, which is not ideal in practice. Therefore, we would like each client to pack η messages into one ciphertext (an RLWE sample). Then the complexity would become $O(\eta^2/N)$, where typically N is chosen larger than 2024 in many FHE applications [14,57,53,20] for the message size is larger than or equal to 2^8. For application scenarios with small number of users ($\eta <$ few thousands), the concrete communication cost would be significantly improved.

Let us assume that the size of each message m_i is a $\log t$ bits integer. The partial decryption d_i consists of $k\eta$ RLWE ciphertexts. Then clients transform each RLWE ciphertext into an LWE ciphertext by viewing an RLWE ciphertext as a vector and rearranging the components, then it runs LWEs-to-RLWE conversion [15] to generate an RLWE ciphertext encrypting $k\eta$ messages (each is embedded into the corresponding coefficient of a message polynomial).

Algorithm 2 Homomorphic placing $\mathsf{HomPlacing}^*$ for the choice of three slots

Input: An encrypted value V_r, A_1, A_2, A_3, where each $A_j := \log \eta$ encrypted bits $\{c_{j,0}, \ldots, c_{j,L-1}\}$, where $L = \log \eta$ and $j \in [3]$.
Output: encrypted boolean value denoting the success/failure of the process.

The server maintains two matrices $L_{\eta \times 3}$ and $I_{\eta \times 3}$
Initialize $L_{i,j} = \mathbf{0}$ and $I_{i,j} = \mathbf{1}$ for all i, j values.
initialize a matrix $z_{i,j} = \mathbf{0}$ for $z_0, \ldots, z_{\eta-1}$ and $j \in \{0, 1, 2\}$
for $d \leftarrow \{0, 1, 2\}$ // number of choices // **do**
    Initialize root of the tree: $b_0 := \mathbf{1}$
    Initialize the node values: $b_i := 0$ for $i \in \{1, \ldots, 2\eta - 2\}$
    for $i \leftarrow 0, \ldots, L - 1$ **do**
        for $j \leftarrow 0, \ldots, 2^i - 1$ **do**
            $b_{2^{i+1}+2j-1} := b_{2^i-1+j} - b_{2^i-1+j} \cdot c_{d,i}$
            $b_{2^{i+1}+2j} := b_{2^i-1+j} \cdot c_{d,i}$
        end for
    end for
    for $i \leftarrow 0, \ldots, \eta - 1$ **do**
        $z_{i,d} := z_{i,d} + b_{2^{L+1}-1+i}$
    end for
end for
// We are done with parsing the indices sent by the client. We want to place the value in the first available empty slot. //
let $\mathsf{hasWritten} := 0$
for $d \leftarrow \{0, 1, 2\}$ **do**
    for $k \leftarrow \{0, 1, 2\}$ // number of slots in each bin // **do**
        for $i \leftarrow 0, \ldots, \eta - 1$ **do**
            $h = z_{i,d} \cdot I_{i,k} \cdot (1 - \mathsf{hasWritten})$
            $L_{i,k} := L_{i,k} + h \cdot V_r$
            $I_{i,k} := I_{i,k} - h$ // Indicating the slot is used //
            $\mathsf{hasWritten} := \mathsf{hasWritten} + h$
        end for
    end for
end for
return hasWritten

Sender Anonymity Property Now we show that our protocol achieves sender anonymity even when all recipients are corrupted (and the server is corrupted). For this section we only consider passive attacks. We discuss about relevant active attacks in Section 6.

The following lemma shows that our shuffle strategy works by mimicking the protocol, but everything under plaintext.

Lemma 1. *Given a matrix of size $\eta \times 3$ (each element initialized to $\perp$), and a total of η incoming elements, suppose each incoming element is placed in the matrix with the following strategy: Three values a_1, a_2, a_3 are chosen uniformly at random from $\{0, \ldots, \eta - 1\}$, and the element is placed (or overwritten) in the position (i, j) such that $i \in \{a_1, a_2, a_3\}$ and i-th column has the most number of $\perp$ among them, and j is the number of $\perp$ in i-th column. After all the incoming elements are placed, the elements of each column are shuffled with a random permutation independent of all other columns. Independent of the order of arrival, the partial order of two arbitrarily chosen elements is uniformly random, given that all elements have been placed successfully.*

Proof. We can show this by induction on the number of elements that has already arrived.

Base Case (matrix size $\eta \times 3$, number of elements $= 2$) When the first element arrives, all the slots were empty. Therefore, the first element would be placed in the first row. Let us assume that index in the first

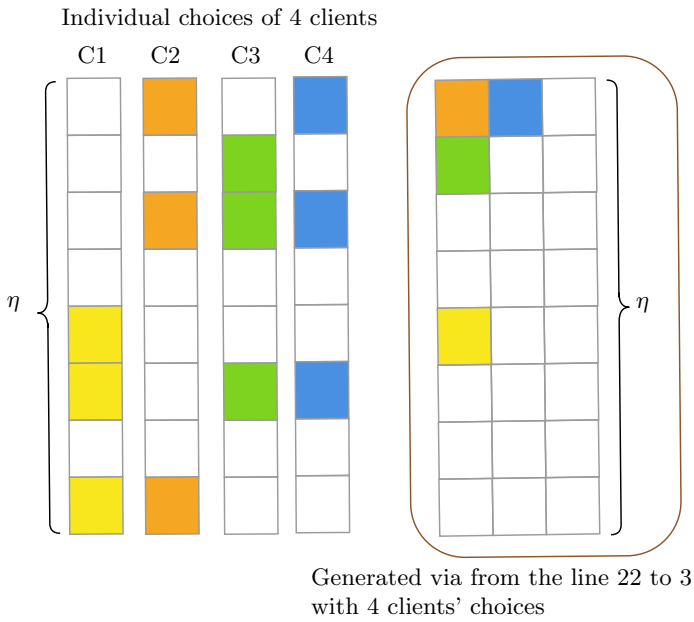

Fig. 3: A high level idea of how Algorithm 2 works in our protocol with visualized arrays. Each client selects its own d slots independently (illustrated in different colored boxes). And the 4 arrays on the left are generated via the loop from the line 7 to 19 for clients independently. If a slot was taken by one of the previous clients' choice, the message for the same slot is moved to other slot via the loop from the line 22 to 31. The algorithm updates DB by taking each client's choice by adding the given messages to the assigned slot of the DB one by one. The $\eta \times 3$ matrix on the right represents the database after updating 4 client's choices. Each color represents the corresponding client's choice of slot and the corresponding message is inserted there.

row is x where $x < \eta$. For the second element, it is possible that $a_1 = x$, however, in that case, the element will be placed in $(a_2, 0)$. The probability of $a_1 < a_2$ is 0.5.

Inductive Hypothesis Let us assume that the lemma holds for any number of elements $< \chi \leq \eta$. We want to show that it holds for χ elements. If either of the chosen two elements is not the χ-th element, the lemma holds (based on inductive hypothesis).

If one of those two elements is the χ-th element: there is a probability that one of $\{a_1, a_2, a_3\}$ collides with the other element. If they are not placed in the same column, the partial order of the two elements is random (the partial order of two random numbers is random). If they are placed in the same column, the shuffle (of the column) step makes their partial order uniformly random.

We ignore the scenarios where a column could overflow (with a very small probability), since we only consider successful placement of all elements.

Using the above lemma, we can show that our protocol achieves sender anonymity. Intuitively, the same shuffle is executed by the server in the actual protocol, but under FHE. And, since the adversary does not know the inputs from the honest senders, the partial ordering of honest messages are unpredictable to the adversary.

Theorem 1 (Sender Anonymity). *Assuming IND-CPA security of the underlying FHE scheme $\mathcal{E}$, and the messages come from a uniform distribution, our protocol sPAR provides sender anonymity as defined in Definition 1 with $\delta \in neg(\lambda)$, where λ is the security parameter.*

Proof. We want to show the following: if there exist an adversary $\mathcal{A}$ that can achieve non-negligible δ in the sender anonymity game $\mathcal{G}_{\mathsf{SA}}^{\Pi,\mathcal{A},\eta}(1^\lambda)$, we can construct an adversary $\mathcal{A}_{FHE}$ that can break IND-CPA security of the underlying FHE scheme $\mathcal{E}$.

In our argument, we use an equivalent modified version of the standard IND-CPA game used to define the security of $\mathcal{E}$: on a given challenge message pair (m_0, m_1), the challenger $\mathcal{C}_{FHE}$ of the encryption system returns a ciphertext pair $c_0 = \mathsf{Enc}_{\hat{\mathsf{pk}}}(m_b)$ and $c_1 = \mathsf{Enc}_{\hat{\mathsf{pk}}}(m_{1-b})$ instead of just one ciphertext $\mathsf{Enc}_{\hat{\mathsf{pk}}}(m_b)$.[4]

The challenger of the anonymity game $\mathcal{C}$ uses the encryption oracle from the IND-CPA game for all encryptions — that ensures the same encryption key in both the games. Recall that, in the sender anonymity game, the adversary controls all the clients except u_0 and u_1, as well as the router/server. If $\mathcal{A}$ uses the challenge pair (u_0, r^*, m_0) and (u_1, r^*, m_1), $\mathcal{A}_{FHE}$ uses the challenge message pair (m_0, m_1). Then $\mathcal{C}$ and $\mathcal{A}$ runs our protocol until $\mathcal{A}$ outputs a guess for the challenge bit. If the adversary $\mathcal{A}$ in the anonymity game returns 0, $\mathcal{A}_{FHE}$ also returns 0.

Here we utilize two facts: (1) since the messages come from a uniform distribution, sorting implies shuffling; (2) And, because of the above, Lemma 1 is applicable here: $\mathcal{A}_{FHE}$ does not learn anything by correlating the order of sending and the final order of messages . That means $\mathcal{A}_{FHE}$ has the advantage of δ over a random coin toss to break IND-CPA game.

4.3 Failure Probability With Choice of Three

In Algorithm 2, only one of the indices is written out of the three chosen by the client: if the first index (encrypted in A_1) cannot be placed in the first segment (each segment of η slots, and 3 segments in total), the second index (encrypted in A_2) is placed in the second half. This maps to the following balls-and-bins problem: there are η bins, and each bin has a capacity of 3; three indices are chosen for each ball and is placed to the bin with lower ball count. And if the two or more bins have the same ball count, the tie is broken by using the first index chosen by the user. In this scenario, the total number of bins is η, and a total of η balls are placed one after another. An extremely low probability that any of the bins has an overflow directly implies an extremely low probability that a message cannot be placed in the list maintained by the server in our protocol.

Based on the analysis from [51], we know that the max load for a bin in such ball-and-bins problem is bounded by $\log\log\eta/\log d + O(1)$ for η bins and d choices for every ball. We refer to Figure 4.1 of [51] for the concrete probability analysis for different loads in the bins. We present a summary in Table 2. In that figure, n in the left column represents the number of balls in a bin, with total #balls = total #bins = η; and d represents the number of choices for each ball. From their analysis, with the choice $d = 3$ and the probability of having 4 balls or more in a bin ($n \geq 4$) is extremely unlikely ($< 10^{-11}$), which directly translates to the server maintain a list with 3η slots in our protocol.

	$d=2$		$d=3$	
	Prediction	Sim.(1M)	Prediction	Sim.(1M)
$n=1$	0.7616	0.7616	0.8231	0.8230
$n=2$	0.2295	0.2295	0.1765	0.1765
$n=3$	0.0089	0.0089	0.00051	0.00051
$n=4$	0.000006	0.000007	$< 10^{-11}$	0
$n=5$	$< 10^{-11}$	0	$< 10^{-11}$	0

Table 2: The concrete probability analysis from [51] for different loads in the bins. $n = i$ in the left column represents the number of balls in a bin, d represents the number of choices for each ball, and value in each cell represents the probability, with #total balls = #total bins.

[4] By defining an appropriate hybrid, this modified game can be easily shown to be equivalent to the standard CPA-security encryption game.

4.4 Modifications for Recipient Anonymity

In order to achieve recipient anonymity, each message can be encrypted with another layer of encryption (and this layer could be a classical public-key encryption, e.g., RSA) under the recipient's public key. Then the underlying messages broadcast by the server can only be revealed to the designated receivers.

Each recipient $r_i \in \mathcal{R}$ creates a public-key $\mathsf{pk}_i^{\mathsf{classical}}$ associated with them, and only r_i knows the corresponding secret key $\mathsf{sk}_i^{\mathsf{classical}}$. Similar to *onion services* in Tor, this public key $\mathsf{pk}_i^{\mathsf{classical}}$ is publicly known but the corresponding physical identity r_i is not known to anyone. [5]

Suppose, the sender u_i wants to send a message m_i to the recipient with public-key $\mathsf{pk}_j^{\mathsf{classical}}$. Then u_i first encrypts the message m_i with $\mathsf{pk}_j^{\mathsf{classical}}$ to generate m_i'. Then u_i computes $c_i = \mathsf{Enc}_{\hat{\mathsf{pk}}}(m_i')$ and sends it to the server.

Then the rest of the protocol remains identical. However, after the decryption phase, the server broadcasts the output $\{m_i'\}_{i\in\{1,\ldots,\eta\}}$ of the decryption phase to all the recipients. Only the intended recipient r_j can decrypt m_i' to retrieve the message m_i.

Theorem 2 (Recipient Anonymity). *Assuming the hardness of Discrete-logarithm problem, our protocol provides recipient anonymity as defined in Definition 2 with $\delta \in neg(\lambda)$, where λ is the security parameter.*

Proof (Proof Sketch). Note that we use straightforward broadcast mechanism to achieve recipient anonymity. Since the server broadcasts the output of FHE decryption to all recipients, there is no intersection-leakage based on the recipient's activities. Additionally, if a corrupted sender (possibly colluding with the server) want to deduce any information from $\mathsf{pk}_i^{\mathsf{classical}}$, that can be reduced to breaking the Discrete-logarithm problem.

Note that, since a compromised sender would already know the message to encrypt, the security of the FHE scheme does not impact recipient anonymity.

5 Implementation And Benchmarks

Our protocol leverages the multi-party version of RLWE encryptions, especially TFHE variants [18,56] which supports every homomorphic operation that we use for our protocol, namely external product. We stress that we do not employ bootstrapping of TFHE, but external product which is a main building block operation for TFHE bootstrapping. We provide an implementation in the Rust Programming Language using the Concrete Core library[6] [19] for homomorphic operations since external product is implemented there efficiently.

5.1 Experimental Setup

To simulate a Server implementing our scheme, we benchmarked our implementation on a dedicated cloud instance running Red Hat Linux. The instance was equipped with an Intel® Xeon™ Platinum 8360Y CPU (36-core@2.4ghz) and 2TB of RAM. To simulate a client, we compiled our code, targeting a Linux PC equipped with an Intel® Core™ i5-6600 CPU (4-core@3.3ghz) and 8GB of RAM. We assume that the clients and the server agree on the FHE parameters prior to execution. Our instantiations use parameters in Table 3, yielding 119 bits of security [2] for the underlying FHE scheme.

5.2 Benchmarks

We benchmark the performance of our design for different number of clients (32, 64, 128) and report the number in Table 4. For each benchmark, we run the experiment 10 times, and take the avg. of those runs.

The most expensive step is the query creation step for sending a message: this includes encrypting the message as well as the binarized indices. The least expensive step (for a small number of clients) in the partial

[5] This could be achieved by using our protocol in Section 4.2 as a sender, and send $\mathsf{pk}_i^{\mathsf{classical}}$ to all the desired parties.
[6] "concrete-core" version: 1.0.2

Table 3: TFHE Parameters

Parameter	Value
standard deviation	2^{-55}
polynomial degree N	2048
decomposition parameters for key switching (g, ℓ)	$(2^5, 11)$
decomposition parameters for query (g, ℓ)	$(2^5, 9)$
plaintext modulus t	2^8
ciphertext modulus q	2^{64}

Table 4: Computation times (in milliseconds) required by different steps of sPAR with 32, 68, and 128 clients in the system.

η	**Client:Send** per message	**Server:Write** per message	**Partial Dec.** η messages	**Sorting** 3 messages
32	2845	918	268	<100
64	3407	1442	544	<100
128	4236	4778	2381	<100

decryption, however, it grows almost linearly with the number of clients. Note that, the computations on the server is highly parallelizable, since a significant portion of the computation for each query can be computed independent of other queries (Lines 3-17 in Algorithm 2). We include sorting time for each bucket in last column of Table 4 using only a single thread for processing. We note that the sorting time for η bucket does not increase much if we can parallelize on η thread. We refer to the Table 5 of [6] for a comprehensive study on the overhead of such sorting mechanisms.

Overall, we can estimate from our benchmarks that each round of our protocol can be completed in less than a few seconds for small scale applications (≤ 128 users). With such overheads and scalability, our protocol could be suitable for application scenarios like mixing cryptocurrency transactions or peer-to-peer messaging in LAN scenarios.

6 Defense Against Active Adversaries

We do not consider malicious servers whose sole purpose is to DoS the system. However, a malicious server might decide to replace one or many ciphertext **after** Server:Write step in combination with equivocation to deanonymize the users. Concretely, the server could launch the following attack: (1) The server ignores the ciphertext sent by all but one client (corresponding to the target user Alice). (2) The server runs the homomorphic placing (with choices) algorithm for the message from Alice. (3) After that the server broadcast the output for the partial decryption phase. (4) The clients allow the server to decrypt the message (and deanonymize Alice) by executing the partial decryption. We aim to defend against such active deanonymization attacks.

6.1 Equivocation Protection

We get equivocation protection almost for free with minor tweaks in the FHE ciphertext structure. Just after the Server:Write step, if the server sends different set of ciphertexts to different clients, the decryption for the non-matching ciphertexts will fail anyway. However, we additionally want the honest clients to be able to detect such attacks from the server.

To that end, we incorporate the history of all past communications to derive the common public-key in each given round. Suppose, $\hat{\mathsf{pk}}$ be the public key agreed during the setup phase. Then the public key used in round r is computed as $\mathsf{pk}^r = \mathsf{pk}^{r-1} + \mathcal{H}(\tau_{r-1})$, where $\mathcal{H}$ is a cryptographic hash function defined over the public key space, and τ is the transcript of all communications sent by the server. And each client locally computes this updated public key without communicating with each other.

This has two-fold implications: (1) this allows us to generate fresh public-key in each round, without any additional communication. (2) If the server sends different set of messages/ciphertexts to different clients, it will lose the ability to decrypt from the next round. That will allow the honest clients to detect any form of equivocation attacks.

We first provide the exact instantiation of our method. Let $\hat{\mathsf{pk}}$ be the common public key of all clients, the sum of all clients' public keys, which everyone generates during the setup phase (especially before the first round of computation). In other words, $\hat{\mathsf{pk}} := (a, a\sum_{i=1}^{p} s_i + e) \in \mathcal{R}_q^2$, which has been stored in local memory of all clients since the beginning, where a is the CRS. After one round, $\hat{\mathsf{pk}}$ is updated to $\hat{\mathsf{pk}}^1 := (a, a(\sum_{i=1}^{p} s_i + H(\tau_0)) + e^1)$, by adding $a \cdot H(\tau_0)$ on the second component of $\hat{\mathsf{pk}}$. Then each client uses it as the common public key for the next round and repeats the same in every round.

We first stress that the common public key generated in that way is a valid key pair as proved in (See Section 3.2 of [4]) due to key-homomorphic property of LWE samples. In other words, summing the second components of two LWE samples where the first component is fixed produces a new LWE encryption of the sum of the plaintexts under the sum of the secret keys. The security of the key is preserved by LWE symmetric key encryption [59].

Now we want to check whether adding $H(\tau)$ for every round would violate the security of the common public key. We start with $\hat{\mathsf{pk}}$ which is indistinguishable from randomly selected samples from $\mathcal{R}_q^2$. Since $\mathcal{H}$ is a cryptographic hash function (a.k.a. random oracle) defined over $\mathcal{R}_q$, therefore, the distribution of $\mathcal{H}(\tau_i)$ is indistinguishable from uniform random distribution over $\mathcal{R}_q^2$. Moreover, a is the CRS chosen uniformly random from $\mathcal{R}_q$, hence adding $a\mathcal{H}(\tau_i)$ to uniformly random looking element of $\hat{\mathsf{pk}}$ does not change its distribution indistinguishable from uniform random distribution over $\mathcal{R}_q^2$. After receiving messages encrypted under the updated public key for any j-th round of computation, each client can run partial decryption with its own secret key $s_i + H(\tau_{j-1})$ (updated accordingly).

6.2 Defense Against Active Attacks Using Verifiable FHE

Verifiable FHE [5,66] allows outsourced computation over encrypted data via FHE to be verified efficiently. Therefore, it allows clients to detect if the server follows the protocol honestly in a reasonably short time. The well-known methods to achieve verifiable FHE are zero-knowledge proof, SNARKs (succinct non-interactive argument of knowledge), using TEEs (Trusted Execution Environment). Recently, there have been practical improvements on verifiable FHE [5,45,64,66] based on RLWE assumption, therefore all of which are compatible with our instantiation.

6.3 Without Verifiable FHE

The technique mentioned above adds significant computation overhead on both the server and the client. However, in several deployment scenarios, this can be done more efficiently. More concretely, we require that the output of a disrupted round can be discarded as long as the honest clients can detect such disruptive behaviors with overwhelming probability, and without impacting any other valid rounds. For instance, protocol like Coinshuffle [60] mixes the a set Blockchain of transactions by shuffling the destination addresses. If the shuffle fails, the clients can discard that round, and regenerate the destination addresses for the transactions. In another example for the DC-net based protocol OrgAn [26], they use the core/base protocol to shuufle random values generated by the clients to derive a partial order, and they utilize that partial order to send real (large) messages with a 'Bulk' protocol. If the core protocol is disrupted, they can simply discard that round, and re-run the protocol with new values. In such scenarios, we propose an efficient way to detect any malicious attacks from the server, and a *reactive* defense mechanism.

Detection Technique To detect against malicious behavior, the protocol requires that the server broadcasts the output message set after a round is over. This output is protected against equivocation attacks, because of the mechanism presented in Section 6.1. That would allow an honest client to detect after the round if their message has been dropped. Then they start a *Verification* protocol.

Verification The verification is a collaborative algorithm among the clients and the server. In this step, each client reveals their messages and the chosen indices to everyone else. That will allow verification of **Client:Encryption()** and **Server:Write()** steps. However, for the **Partial Decryption** step, we do not want the clients to reveal their secret keys. So, the clients generate verifiability proof for this step as mentioned in Section 6.2, and broadcasts them. The server flags any proof from a client that does not match with the original ciphertext. The server broadcasts the proof as well as the original ciphertext to all the clients.[7] Each client can locally verify that the server did not corrupt the final computation. If the server cannot produce any disruptive client or prove correctness for its computations, the server is considered as the culprit.

Note that the verification mechanism is reactive: this step is only run when there are disruptions; and when there are no disruptions in the protocol, it does not significantly impact the performance of the shuffle protocol.

With very small probability, some messages might get corrupted because of the collision issue from Section 4.3. However, the verification step still would allow an honest server to prove the correctness of computation; and the reason for corruption can be easily verified from the values revealed by the clients.

6.4 Heuristic Defense With Many Honest Clients

We can avoid revealing the messages (and thus, requiring the round to be discarded) by assuming that only a fraction of clients could be corrupted, and all clients are available for the partial decryption step. We modify the protocol in the following way so that, if the server attempts to block messages from even one honest client, with high probability many other honest messages will be disrupted.

Modifications in Protocol In the client encryption step, instead of sending one ciphertext, the client u_i generates ν ciphertexts $c_{i,1}, \ldots, c_{i,\nu}$. Only one of them encrypts the message m_i and rest of them encrypts random values chosen by the client. So, for a randomly chosen value $1 \leq x \leq \nu$, the client sets: $c_{i,x} = \mathsf{Global.Enc}_{\hat{\mathsf{pk}}}(m_i)$; and $c_{i,j} = \mathsf{Global.Enc}_{\hat{\mathsf{pk}}}(r_{i,j})$ for all $j \neq x$ and $r_{i,j} \xleftarrow{\$} R_t$. The client additionally sends the encryption of x: $C_i = \mathsf{Global.Enc}_{\hat{\mathsf{pk}}}(x)$ to the server. Note that the value of ν can be independent of η; however, for performance we want $\nu << \eta$. We explain the impact of ν on security and performance shortly. For simplicity of explanation, we explain the rest of protocol with $\nu = 2$.

Server Computation. The server computation is also modified such that the $r_{i,j}$ values from u_i are additively placed in all slots except the three slots chosen by the client. With $\nu = 2$, C_i is an encryption of one bit information: $x \in \{0, 1\}$. We provide the computation steps in Algorithm 3. At the end of the computation with all the input values from all the clients, each slot will contain either a message with some random values added to it or just the sum of a set of random values.

Partial decryption. During the partial decryption step, the client u_i with secret key s_i executes the following computations: $d_i = \mathsf{Partial.Dec}(s_i, c_t) - \sum_{1 \leq j \leq \nu} \Delta \cdot r_{i,j}$ for a index t when $t \notin \{a_1, a_2, a_3\}$; otherwise, $d_i = \mathsf{Partial.Dec}(s_i, c_t)$. Since the server does not know the value of x used by a client, the server attempting to drop messages from a client will drop one of the $r_{i,j}$ values from that client with probability $\frac{\nu-1}{\nu}$. However, that client will still subtract $\Delta \cdot r_{i,j}$ from all excepts her three chosen slots. Therefore, the adversarial server will inadvertently disrupt all the messages in the $(\eta - 3)$ buckets. Assuming there are many honest clients in the system, the server will deal with verification request from many clients, and the identity of the targeted client would still be protected among many other clients.

[7] Requiring a digital signature with any ciphertexts from clients would allow the server to prove that the ciphertext indeed came from the client.

Algorithm 3 Active attack resistance.

Input: Encrypted values c_1, c_2 and C; A_1, A_2, A_3, where each $A_j := \log \eta$ encrypted bits $\{c_{j,0}, \ldots, c_{j,L-1}\}$, where $L = \log \eta$.
Same as steps 1-20 in Algorithm 2.
hasWritten := 0
for $d \leftarrow \{0, 1, 2\}$ **do**
  for $k \leftarrow \{0, 1, 2\}$ // number of slots in each bin // **do**
    for $i \leftarrow 0, \ldots, \eta - 1$ **do**
      $h = z_{i,d} \cdot I_{i,k} \cdot (1 - \text{hasWritten})$
      $L_{i,k} := L_{i,k} + h \cdot (c_1 \cdot (\mathbf{1} - C) + c_2 \cdot C)$
        $+ (\mathbf{1} - z_{i,d}) \cdot (c_1 \cdot C + c_2 \cdot (\mathbf{1} - C))$
      $I_{i,k} := I_{i,k} - h$
      hasWritten := hasWritten $+ h$
    end for
  end for
end for
return hasWritten

7 Extensions And Discussions

7.1 $IND - CPA^D$ Attack

Since we are using threshold decryption protocol [54] as almost all multiparty/multi-key FHE schemes employ, it would be vulnerable to $IND - CPA^D$ attack [16] where a passive adversary who can access to the decrypted value and the corresponding ciphertext might have a high chance to extract secret key.

However, with the technique addressed in Section 6.4, we can avoid a special decryption oracle which cause the attack. In our case, the corresponding message of the given ciphertext is never revealed due to the randomness (which is always kept private) added to the message, therefore a decryption oracle does not exist.

We note that this technique is not an universal countermeasure for any threshold FHE. However, it can be an efficient countermeasure for such threshold FHE applications which only reorder or shuffle the messages, not changing message values (e.g. computation on messages) with multiple decryptors.

7.2 Client Churn And Threshold Version

The protocol specification is Section 4 does not consider the scenario where some clients might become inactive/unavailable in some rounds. If such a scenario occurs, the parties will be unable to retrieve the messages, since each client needs to provide their partial decryption. Without any major modification in the protocol, it is possible to use *Threshold Homomorphic Encryption* [4]: as long as a threshold number of clients are available, the decryption step would be possible. They can even decide to decrypt even when all the clients do not send their messages. We consider the specific nuances (how to detect an inactive client, the quantification of anonymity when some clients do not send messages, possible performance optimization) of the threshold version as future work.

7.3 Limitations And Conclusion

We consider our work as a second step towards designing anonymous messaging systems without any trusted infrastructure. Our work has followed the footsteps of Shi and Wu [62] that showed the theoretical possibility of such systems. We do not attempt to achieve high scalability of Tor [30] or Mixnets [29,58], not the performance of some MPC-based systems [32,31]. Still, our proof-of-concept implementation demonstrates that our design is implementable, and suitable for small scale applications with few hundred users where the application can tolerate a few seconds of delay in message delivery. Our design strategy shows a new direction for designing anonymous messaging systems using FHE.

Acknowledgment

This work was started when Debajyoti Das was at KU Leuven, and continued during his time at Umeå University. This work has been partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. Any opinions, findings and conclusions expressed in this material are those of the authors and do not necessarily reflect the views of any of the funding agencies.

References

1. Abraham, I., Pinkas, B., Yanai, A.: Blinder – scalable, robust anonymous committed broadcast. In: Proceedings of the 2020 ACM SIGSAC CCS. p. 1233–1252 (2020)
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. Journal of Mathematical Cryptology 9(3), 169–203 (2015), https://doi.org/10.1515/jmc-2015-0016
3. Asharov, G., Jain, A., López-Alt, A., Tromer, E., Vaikuntanathan, V., Wichs, D.: Multiparty computation with low communication, computation and interaction via threshold FHE. pp. 483–501 (2012)
4. Asharov, G., Jain, A., Wichs, D.: Multiparty computation with low communication, computation and interaction via threshold FHE. Cryptology ePrint Archive, Paper 2011/613 (2011), https://eprint.iacr.org/2011/613
5. Atapoor, S., Baghery, K., Pereira, H.V.L., Spiessens, J.: Verifiable FHE via lattice-based SNARKs 1(1), 24 (2024)
6. Azogagh, S., Killijian, M.O., Larose-Gervais, F.: A non-comparison oblivious sort and its application to private k-NN. Cryptology ePrint Archive, Paper 2024/1894 (2024), https://eprint.iacr.org/2024/1894
7. Backes, M., Kate, A., Manoharan, P., Meiser, S., Mohammadi, E.: Anoa: A framework for analyzing anonymous communication protocols. In: 2013 IEEE 26th Computer Security Foundations Symposium. pp. 163–178. IEEE Computer Society, New Orleans, LA, USA (2013), https://doi.org/10.1109/CSF.2013.18
8. Barman, L., Dacosta, I., Zamani, M., Zhai, E., Pyrgelis, A., Ford, B., Feigenbaum, J., Hubaux, J.: Prifi: Low-latency anonymity for organizational networks. Proc. Priv. Enhancing Technol. 2020(4), 24–47 (2020)
9. Bonneau, J., Clark, J., Kroll, J.A., Miller, A., Narayanan, A.: Mixcoin: Anonymity for bitcoin with accountable mixes. In: Proceedings of Financial Cryptography and Data Security (FC'14) (March 2014)
10. Bonte, C., Iliashenko, I., Park, J., Pereira, H.V.L., Smart, N.P.: FINAL: Faster FHE instantiated with NTRU and LWE. pp. 188–215 (2022)
11. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. In: ITCS '12 (2012)
12. Chaum, D.: The dining cryptographers problem: Unconditional sender and recipient untraceability 1(1), 65–75 (Jan 1988)
13. Chaum, D., Javani, F., Kate, A., Krasnova, A., Ruiter, J., Sherman, A.T., Das, D.: cmix: Anonymization by high-performance scalable mixing. Tech. rep., Technical report (2016), http://www.cs.bham.ac.uk/~deruitej/papers/cmix.pdf
14. Chen, H., Chillotti, I., Ren, L.: Onion ring ORAM: Efficient constant bandwidth oblivious RAM from (leveled) TFHE. pp. 345–360 (2019)
15. Chen, H., Dai, W., Kim, M., Song, Y.: Efficient homomorphic conversion between (ring) LWE ciphertexts. pp. 460–479 (2021)
16. Cheon, J.H., Choe, H., Passelègue, A., Stehlé, D., Suvanto, E.: Attacks against the ind-cpad security of exact fhe schemes. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 2505–2519. CCS '24, Association for Computing Machinery, New York, NY, USA (2024), https://doi.org/10.1145/3658644.3690341
17. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. pp. 409–437 (2017)
18. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus 33(1), 34–91 (Jan 2020)
19. Chillotti, I., Joye, M., Ligier, D., Orfila, J.B., Tap, S.: Concrete: Concrete operates on ciphertexts rapidly by extending tfhe. In: WAHC 2020–8th Workshop on Encrypted Computing & Applied Homomorphic Cryptography. vol. 15 (2020)
20. Cong, K., Das, D., Park, J., Pereira, H.V.: Sortinghat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. p. 563–577. CCS '22, Association for Computing Machinery, New York, NY, USA (2022), https://doi.org/10.1145/3548606.3560702

21. Cong, K., Geelen, R., Kang, J., Park, J.: Revisiting oblivious top-k selection with applications to secure k-nn classification. In: Selected Areas in Cryptography – SAC 2024: 31st International Conference, Revised Selected Papers, Part I. p. 3–25 (2025), https://doi.org/10.1007/978-3-031-82852-2_1
22. Corrigan-Gibbs, H., Boneh, D., Mazières, D.: Riposte: An anonymous messaging system handling millions of users. In: IEEE Symposium on Security and Privacy. pp. 321–338 (2015)
23. Corrigan-Gibbs, H., Ford, B.: Dissent: Accountable Anonymous Group Messaging. In: Proc. ACM SIGSAC CCS. pp. 340–350 (2010)
24. Corrigan-Gibbs, H., Wolinsky, D.I., Ford, B.: Proactively Accountable Anonymous Messaging in Verdict. In: Proc. 22nd USENIX Security Symposium. pp. 147–162 (2013)
25. Danezis, G., Diaz, C.: A survey of anonymous communication channels. Tech. Rep. MSR-TR-2008-35 (February 2008), https://www.microsoft.com/en-us/research/publication/a-survey-of-anonymous-communication-channels/
26. Das, D., Mangipudi, E., Kate, A.: Organ: Organizational anonymity with low latency. Proceedings on Privacy Enhancing Technologies 2022, 582–605 (07 2022)
27. Das, D., Meiser, S., Mohammadi, E., Kate, A.: Anonymity trilemma: Strong anonymity, low bandwidth overhead, low latency - choose two. In: 2018 IEEE Symposium on Security and Privacy (SP). pp. 108–126. IEEE Computer Society, San Francisco, California, USA (2018)
28. Das, D., Meiser, S., Mohammadi, E., Kate, A.: Comprehensive anonymity trilemma: User coordination is not enough. Proceedings on Privacy Enhancing Technologies 2020(3), 356–383 (2020)
29. Das, D., Meiser, S., Mohammadi, E., Kate, A.: Divide and funnel: a scaling technique for mix-networks. Cryptology ePrint Archive, Paper 2021/1685 (2021), https://eprint.iacr.org/2021/1685, https://eprint.iacr.org/2021/1685
30. Dingledine, R., Mathewson, N., Syverson, P.F.: Tor: The second-generation onion router. pp. 303–320 (2004)
31. Eskandarian, S., Boneh, D.: Clarion: Anonymous communication from multiparty shuffling protocols. Cryptology ePrint Archive (2021)
32. Eskandarian, S., Corrigan-Gibbs, H., Zaharia, M., Boneh, D.: Express: Lowering the cost of metadata-hiding communication with cryptographic privacy. pp. 1775–1792 (2021)
33. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144 (2012), https://eprint.iacr.org/2012/144
34. Golle, P., Juels, A.: Dining cryptographers revisited. In: Proc. of Eurocrypt 2004 (2004)
35. Heilman, E., AlShenibr, L., Baldimtsi, F., Scafuro, A., Goldberg, S.: Tumblebit: An untrusted bitcoin-compatible anonymous payment hub (01 2017)
36. Huete Trujillo, D.L., Ruiz-Martínez, A.: Tor hidden services: A systematic literature review. Journal of Cybersecurity and Privacy 1(3), 496–518 (2021), https://www.mdpi.com/2624-800X/1/3/25
37. Jia, Y., Madathil, V., Kate, A.: HomeRun: High-efficiency oblivious message retrieval, unrestricted. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (2024)
38. Kate, A., Zaverucha, G.M., Goldberg, I.: Pairing-based onion routing with improved forward secrecy. ACM Transactions on Information and System Security (TISSEC) 13(4), 1–32 (2010)
39. Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., Schwarz, M., Yarom, Y.: Spectre attacks: Exploiting speculative execution. In: 2019 IEEE Symposium on Security and Privacy (SP). pp. 1–19 (2019)
40. Kuhn, C., Beck, M., Strufe, T.: Breaking and (Partially) Fixing Provably Secure Onion Routing. In: Proceedings of the 41st IEEE Symposium on Security and Privacy. pp. 168–185. IEEE (2020)
41. Kwon, A., Lu, D., Devadas, S.: XRD: scalable messaging system with cryptographic privacy. In: Bhagwan, R., Porter, G. (eds.) 17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020. pp. 759–776. USENIX Association, Santa Clara, CA, USA (2020), https://www.usenix.org/conference/nsdi20/presentation/kwon
42. Langowski, S., Servan-Schreiber, S., Devadas, S.: Trellis: Robust and scalable metadata-private anonymous broadcast. Cryptology ePrint Archive, Paper 2022/1548 (2022)
43. Lazar, D., Gilad, Y., Zeldovich, N.: Karaoke: Distributed private messaging immune to passive traffic analysis. In: 13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18). pp. 711–725 (2018)
44. Lazar, D., Gilad, Y., Zeldovich, N.: Yodel: Strong metadata security for real-time voice calls. In: 12th Workshop on Hot Topics in Privacy Enhancing Technologies (HotPETs 2019) (2019), https://petsymposium.org/2019/files/hotpets/yodel-final.pdf
45. Lee, J., Cho, S., Kim, S., Park, S.: Verifiable computation over encrypted data via mpc-in-the-head zero-knowledge proofs. International Journal of Information Security 24(1), 1–15 (2025)

46. Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Fogh, A., Horn, J., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., Hamburg, M.: Meltdown: Reading kernel memory from user space. In: 27th USENIX Security Symposium (USENIX Security 18). pp. 973–990. USENIX Association, Baltimore, MD (Aug 2018), https://www.usenix.org/conference/usenixsecurity18/presentation/lipp
47. LópezAlt, A., Tromer, E., Vaikuntanathan, V.: On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In: Proceedings of the forty-fourth annual ACM symposium on Theory of computing. pp. 1219–1234. ACM (2012)
48. Lu, D., Yurek, T., Kulshreshtha, S., Govind, R., Kate, A., Miller, A.: Honeybadgermpc and Asynchromix: Practical asynchronous mpc and its application to anonymous communication. In: Proceedings of the 2019 ACM SIGSAC CCS. pp. 887–903 (2019)
49. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. pp. 1–23 (2010)
50. Madathil, V., Scafuro, A., Seres, I.A., Shlomovits, O., Varlakov, D.: Private signaling. In: 31st USENIX Security Symposium (USENIX Security 22) (2022)
51. Mitzenmacher, M.: The power of two choices in randomized load balancing. IEEE Transactions on Parallel and Distributed Systems 12(10), 1094–1104 (2001)
52. Mouchet, C., Troncoso-Pastoriza, J.R., Bossuat, J.P., Hubaux, J.P.: Multiparty homomorphic encryption from ring-learning-with-errors 2021(4), 291–311 (Oct 2021)
53. Mughees, M.H., Chen, H., Ren, L.: OnionPIR: Response efficient single-server PIR. pp. 2292–2306 (2021)
54. Mukherjee, P., Wichs, D.: Two round multiparty computation via multi-key FHE. pp. 735–763 (2016)
55. Park, J.: Homomorphic encryption for multiple users with less communications. IEEE Access 9, 135915–135926 (2021)
56. Park, J., Rovira, S.: Efficient tfhe bootstrapping in the multiparty setting. IEEE Access 11, 118625–118638 (2023)
57. Park, J., Tibouchi, M.: SHECS-PIR: Somewhat homomorphic encryption-based compact and scalable private information retrieval. pp. 86–106 (2020)
58. Piotrowska, A.M., Hayes, J., Elahi, T., Meiser, S., Danezis, G.: The loopix anonymity system. In: 26th {USENIX} Security Symposium ({USENIX} Security 17). pp. 1199–1216 (2017)
59. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. Journal of the ACM (JACM) 56(6), 34 (2009)
60. Ruffing, T., Moreno-Sanchez, P., Kate, A.: P2P Mixing and Unlinkable Bitcoin Transactions (2017)
61. Sasy, S., Goldberg, I.: Sok: Metadata-protecting communication systems. Proceedings on Privacy Enhancing Technologies (2024)
62. Shi, E., Wu, K.: Non-interactive anonymous router. pp. 489–520 (2021)
63. Shirazi, F., Simeonovski, M., Asghar, M.R., Backes, M., Diaz, C.: A survey on routing in anonymous communication protocols. ACM Comput. Surv. 51(3) (Jun 2018), https://doi.org/10.1145/3182658
64. Thibault, L.T., Walter, M.: Towards verifiable FHE in practice: Proving correct execution of TFHE's bootstrapping using plonky2. Cryptology ePrint Archive, Paper 2024/451 (2024), https://eprint.iacr.org/2024/451
65. Vadapalli, A., Storrier, K., Henry, R.: Sabre: Sender-anonymous messaging with fast audits. pp. 1953–1970 (2022)
66. Viand, A., Knabenhans, C., Hithnawi, A.: Verifiable fully homomorphic encryption (2023), https://arxiv.org/abs/2301.07041
67. Wolinsky, D.I., Corrigan-Gibbs, H., Ford, B., Johnson, A.: Dissent in Numbers: Making Strong Anonymity Scale. In: 10th USENIX OSDI'12. pp. 179–182 (2012)